MERN STACK · INTERVIEW PREP

5 MERN Stack Interview Questions Every Fresher Should Know

MERN Stack Interview Prep Guide

MongoDB | Express.js | React.js | Node.js | Architecture

Devika Jangid
MERN Stack Developer

FREE RESOURCE

01 EXPRESS.JS

What is the difference between `res.send()` and `res.json()` in Express.js?

SHORT ANSWER

res.send() can send strings, HTML, buffers, or objects — Express decides the content type automatically. **res.json()** always converts the response into a JSON string and explicitly sets the `Content-Type` header to `application/json`.

DETAILED EXPLANATION

Both methods end the request-response cycle, but they behave differently depending on what you pass in:

- **res.send(data)** is flexible. If you pass a string, it sets `Content-Type` to `text/html`. If you pass an object or array, Express internally calls `JSON.stringify()` for you — so it "just works" for objects too.
- **res.json(data)** is explicit and predictable. It always stringifies the payload and always sets the header to `application/json`, regardless of what you pass — even a string or number gets wrapped as valid JSON.

In real-world REST APIs, most developers prefer `res.json()` for API responses because it removes ambiguity about the response format — which matters a lot when the frontend (React) is expecting consistent JSON to parse.

EXAMPLE

```
app.get('/api/user', (req, res) => {
  // Both work, but res.json() is clearer for APIs
  res.json({ id: 1, name: 'Devika' });

  // res.send() would also work here, but Content-Type
  // detection is implicit rather than guaranteed
});
```

💡 **INTERVIEW TIP**

Mention that `res.json()` is safer for building REST APIs because it guarantees the `Content-Type` header — interviewers like hearing that you think about consistency, not just "it works."

02 MONGODB

What is the `_id` field in MongoDB, and how is it different from a primary key in SQL?

SHORT ANSWER

Every MongoDB document automatically gets a unique **_id** field of type **ObjectId** if you don't provide one yourself. Unlike an SQL primary key (usually an auto-incrementing number), an ObjectId is a 12-byte value that encodes the creation timestamp, making it globally unique without needing a central counter.

DETAILED EXPLANATION

In SQL databases, primary keys are often simple auto-incrementing integers generated by the database engine in sequence. This works well on a single server, but becomes tricky in distributed systems.

- MongoDB's default **ObjectId** is generated on the client or driver side, not the database — so multiple servers can create unique IDs at the same time without collisions.
- An ObjectId embeds a timestamp, meaning you can extract the approximate creation time of a document directly from its `_id` — useful for sorting by creation order without a separate field.
- You can also assign your own custom value as `_id` (like an email or SKU code) as long as it stays unique within the collection.

This design fits MongoDB's distributed, horizontally-scalable nature — it doesn't rely on one central place to hand out the "next number."

EXAMPLE

```
// Example document stored in MongoDB
{
  _id: ObjectId("66f1a2c9e4b0a1234567890a"),
  name: "Devika Jangid",
  role: "MERN Developer"
}

// Extracting creation time from the ObjectId
const createdAt = doc._id.getTimestamp();
```

💡 **INTERVIEW TIP**

If asked "can two documents ever get the same _id?" — say ObjectId combines timestamp, machine identifier, process ID, and a counter, which makes collisions practically impossible even across multiple servers.

03 REACT.JS

Why does `useEffect` take a dependency array, and what happens if you leave it out?

SHORT ANSWER

The dependency array tells React **when** to re-run the effect. If you omit it entirely, the effect runs after **every single render**. An empty array `[]` means "run once, after the first render." Including variables means "re-run whenever any of these values change."

DETAILED EXPLANATION

`useEffect` lets components perform side effects — API calls, subscriptions, timers, DOM updates — outside the normal rendering flow. React needs to know when it's actually necessary to re-run that code, and the dependency array is how you tell it.

- **No array at all** → the effect runs after every render, including re-renders caused by unrelated state changes. This can cause unnecessary API calls or performance issues.
- **Empty array []** → the effect runs only once, right after the component first mounts — commonly used for an initial data fetch.
- **[value1, value2]** → the effect re-runs only when one of the listed values changes between renders.

A common fresher mistake is forgetting a dependency that's actually used inside the effect, which can lead to stale data being used — this is often flagged by the `exhaustive-deps` ESLint rule.

EXAMPLE

```
useEffect(() => {
  // Runs once, when the component first mounts
  fetchUserProfile(userId);
}, [userId]); // re-runs only if userId changes
```

💡 **INTERVIEW TIP**

If asked to debug an "infinite API calls" bug, check the dependency array first — a missing array, or an object/array recreated on every render inside it, is the most common cause.

04 NODE.JS

Node.js is single-threaded — so how does it handle many requests at once?

SHORT ANSWER

Node.js runs your JavaScript on a **single main thread**, but it offloads slow operations — like file reads, database queries, and network calls — to the **libuv thread pool and the OS**. The **event loop** picks up completed operations and runs their callbacks, so the main thread is never blocked waiting.

DETAILED EXPLANATION

The key idea is the difference between the **call stack** (single-threaded, runs your code) and everything else Node uses to handle I/O:

- When you call something like `fs.readFile()` or a database query, Node doesn't wait around — it hands the task off to the system (via libuv) and immediately moves to the next line of code.
- Once that background task finishes, its callback is placed in a queue.
- The **event loop** constantly checks: "Is the call stack empty? If yes, take the next callback from the queue and run it."

This is why Node.js handles thousands of concurrent connections efficiently for I/O-heavy workloads (like typical REST APIs) — it isn't spinning up a new thread per request like some older server models. However, it's important to note this model isn't ideal for CPU-heavy tasks (like image processing), since those tasks do run on the main thread and can block the event loop unless offloaded to worker threads.

EXAMPLE

```
console.log('1. Request received');

fs.readFile('data.json', (err, data) => {
  // This runs LATER, once the file read finishes
  console.log('3. File data ready');
});

console.log('2. Sent response to next request');
// Output order: 1, 2, 3 — main thread never waited
```

💡 **INTERVIEW TIP**

Always mention the event loop AND the libuv thread pool together — many freshers only mention "the event loop" and skip explaining who actually does the I/O work in the background.

05 MERN ARCHITECTURE

Can you walk through what happens when a user submits a form in a MERN application?

SHORT ANSWER

The request flows through all four layers in order: **React** (captures form data and sends an HTTP request) → **Express** (routes the request and runs middleware) → **Node.js** (the runtime executing your Express server) → **MongoDB** (stores or retrieves the data) — then the response travels back up the same chain to update the UI.

DETAILED EXPLANATION

This question tests whether you understand how the four MERN technologies actually fit together, not just what each one does individually:

- **React (Client):** Captures the form input in component state, then calls an API using `fetch` or `axios` on form submit — typically a `POST` request with a JSON body.
- **Express (Framework):** Receives the request on a defined route, runs middleware (like body parsing, authentication, or validation), and calls the matching controller function.
- **Node.js (Runtime):** The JavaScript engine actually executing your Express server — the environment that makes server-side JavaScript possible.
- **MongoDB (Database):** The controller uses a driver like Mongoose to insert or query a document, then returns the result back to Express.

Finally, Express sends a JSON response back to React, which updates the UI to show a success message or the newly created item.

EXAMPLE

```
// 1. React sends the request
axios.post('/api/users', { name, email });

// 2. Express route + controller (runs on Node.js)
app.post('/api/users', async (req, res) => {
  // 3. Mongoose saves the document to MongoDB
  const user = await User.create(req.body);
  res.status(201).json(user); // 4. Response sent back
});
```

💡 **INTERVIEW TIP**

Draw or describe this flow as a simple diagram in the interview — "Client → Server → Database → back to Client." Interviewers love candidates who can visualise the full request lifecycle, not just one layer.